

Soda Can Masking and IMU Regression Experiments

Generated in /Users/anders/projects/thrash

June 2, 2026

1 Data and Common Setup

The source video was `detail.MOV`, an iPhone MOV with a video stream and a per-frame metadata stream used as IMU-like orientation data. The IMU rotation targets were read from stream `0:4` as 144-byte records, using float channels at offsets 20, 22, and 23 for X/Y/Z rotation. Most later experiments used the first 10 seconds at 60 fps, giving 600 samples.

The visual preprocessing evolved from direct dark-component soda-can masks to a high-quality full-resolution crop:

- `soda_can_mask_ffnn_imu_out/soda_can_mask_only.mp4`: 512x512 masked crop at 10 fps.
- `soda_can_mask_ffnn_imu_heat16_out`: same idea with 16 heat-equation iterations before masking/training.
- `fullres_gaussian_filter_benchmark_out/fullres_soda_can_masked_crop_99pct.mp4`: 2160x2160, 60 fps, 10 seconds, H.264 CRF 1.

2 Mask and IMU Prediction Videos

The initial FFNN experiments trained a 32-hidden-neuron model on 512x512 masked soda-can crops to regress IMU XYZ rotation deltas. A prediction-overlay video was generated with reference and predicted IMU values burned into each frame.

Run	Train time	Training MSE	Overlay MSE
32 hidden, RAdam	300 s	0.000978603	0.00816613
32 hidden, heat16, RAdam	300 s	0.00122747	0.00809223
32 hidden, heat16, damped Gauss–Newton	600 s	0.0000656933	0.00714757
64 hidden, heat16, L-BFGS	600 s	0.0225186	0.0266362

The damped diagonal Gauss–Newton/LM update was the strongest of these masked-crop training runs. Raw stochastic L-BFGS from random initialization was unstable, so it was rerun with a RAdam warm start; even then it underperformed.

3 Architecture Sweep on Grayscale Mask Frames

Plain gradient descent was tested on grayscale masked frames using different small FFNN architectures. Each model used batch size 32.

3.1 10 Seconds per Architecture

Architecture	Eval MSE
2x8	0.0199017
1x16	0.0219493
1x8	0.0229684
1x4	0.026123
1x32	0.0270697
2x16	0.0283049
2x4	0.02843
2x32	0.0326764
1x128	0.0799805
1x2	0.115355
1x256	0.385466

3.2 60 Seconds per Architecture

Architecture	Eval MSE
1x4	0.00763605
2x8	0.00783951
1x8	0.0101736
2x16	0.0114428
1x16	0.0118503
2x4	0.0128558
2x32	0.0135664
1x32	0.0178537
1x2	0.0190964
1x128	0.0364814
1x256	0.0509232

The longer run changed the winner from 2x8 to 1x4. Large dense models were consistently disadvantaged under plain GD because their first layer made each update expensive, so they completed far fewer batches.

4 Line Probe RAdam Benchmark

For each $n = 1, \dots, 32$, inputs were restricted to n random horizontal lines plus one random vertical line from each 512x512 masked frame. A 32-hidden-neuron FFNN was trained with RAdam for 10 seconds per n , batch size 32.

The best values were tightly clustered:

n	Eval MSE
11	0.00486113
2	0.00486229
7	0.00486563
26	0.00486652
1	0.00486791

The worst result was $n = 24$, with 0.00503475. The narrow spread shows that sparse line probes were already enough to recover much of the pose-correlated signal in this short benchmark.

5 Gaussian Radial Information Filters

A 2160x2160, 60 fps, 10 second full-resolution masked crop video was generated at CRF 1. Training used 512x512 downsampled frames. A centered Gaussian radial information filter was applied before training:

$$w(x, y) = \exp \left(-\frac{(x - c_x)^2 + (y - c_y)^2}{2\sigma^2} \right)$$

with $\sigma = \text{width}/d$, for $d \in \{2, 4, 8, 16, 32\}$. Each model was a 32-hidden-neuron FFNN, batch 64, RAdam, 30 seconds.

Filter	Eval MSE
w/2	0.00210284
w/8	0.00355526
w/32	0.0115121
w/4	0.0158287
w/16	0.0162277

The widest Gaussian, $w/2$, worked best. It preserved broad silhouette and label information that was useful for pose regression.

6 Full-Batch Dense FFNN

The best Gaussian filter, $w/2$, was then combined with a 64-hidden-neuron dense FFNN. Training used all 600 samples as a full batch with RAdam for 600 seconds.

Metric	Value
Samples	600
Hidden neurons	64
Full-batch steps	266
Final batch MSE	1.06319×10^{-5}
Eval MSE	1.04145×10^{-5}

This was the strongest numerical result in the whole sequence. Full-batch optimization gave a smooth, stable error curve and avoided minibatch noise.

7 Classic Small CNN Test

A small classic CNN was tested on the same $w/2$ Gaussian-filtered data for 600 seconds full-batch. To keep the equivalent capacity near the requested 32-neuron scale and stay cache-friendly, the input was average pooled from 512x512 to 32x32, followed by a 4-channel 3x3 convolution, a stride-2 8-channel 3x3 convolution, global average pooling, and a linear XYZ head.

Metric	Value
Samples	600
Full-batch steps	1577
Final batch MSE	0.00806296
Eval MSE	0.00805397

The CNN was stable and eventually improved after a long plateau, but it was much worse than the dense full-batch FFNN. The likely reason is that the target is global phone/camera orientation rather than local spatial class structure; the dense model can directly use global appearance and silhouette correlations, while the small CNN discards too much absolute spatial detail through pooling and weight sharing.

8 Main Conclusions

1. Full-batch training on the 600-frame, $w/2$ -filtered high-definition masked crop set was the best-performing setup.
2. The 64-hidden dense FFNN with full-batch RAdam reached 1.04×10^{-5} eval MSE, far better than minibatch and small-CNN alternatives.
3. Damped Gauss-Newton/LM was effective on the 32-hidden masked-crop setup, while L-BFGS was noisy and underperformed.
4. Wider Gaussian radial filters performed better than narrow center-only filters, suggesting pose information is spread across the can silhouette and label.
5. Sparse line probes were surprisingly competitive in short runs, with $n = 11$, $n = 2$, and $n = 7$ nearly tied.
6. Very large dense models under plain GD were inefficient under fixed time budgets because they completed too few updates.

9 Key Artifacts

- `gaussian_w2_fullbatch64_600_out/w2_fullbatch64_radam_error.svg`
- `gaussian_w2_cnn32_fullbatch_600_out/cnn32_fullbatch_error.svg`
- `fullres_gaussian_filter_benchmark_out/fullres_soda_can_masked_crop_99pct.mp4`
- `ffnn_arch_benchmark_60s_out/all_architectures_error.svg`
- `line_probe_radam_benchmark_out/line_probe_all_errors.svg`